



BÀI TEST KỸ NĂNG VÒNG 2 & KHUNG ĐÁNH GIÁ NĂNG LỰC

VỊ TRÍ: THỰC TẬP SINH FULLSTACK DEVELOPER (NODE.JS + REACT.JS)

1. THÔNG TIN CHUNG & QUY CHẾ LÀM BÀI

Chủ đề: Bài Test Kỹ Năng Vòng 2 – Thực Tập Sinh Fullstack Developer.

Thời gian làm bài: 36 giờ.

Hình thức nộp: GitHub Repository public và link Demo nếu có.

Ứng viên có thể lựa chọn công nghệ phù hợp với năng lực.

- **Frontend:** React.js được khuyến nghị.
- **Backend:** Node.js với Express / Fastify / NestJS.
- **Database:** MySQL / PostgreSQL / MongoDB hoặc công nghệ tương đương.

2. BỐI CẢNH DỰ ÁN & ĐỀ BÀI

Công ty TNHH Công Nghệ và Giáo Dục PKS hiện vận hành hai thương hiệu gồm PKS Education, tập trung vào đào tạo thực chiến Co-op IT và Tin học quốc tế MOS, và PKS Technology, tập trung vào giải pháp công nghệ, Web chuẩn SEO và Mini-ERP.

Đề bài yêu cầu ứng viên xây dựng mini web-app “PKS Course & Enrollment Portal” phục vụ tra cứu, quản lý và ghi danh các khóa học công nghệ. Hệ thống gồm hai vai trò chính là Học viên (Student) và Quản trị viên (Admin/Staff).

3. YÊU CẦU CHỨC NĂNG CHI TIẾT

3.1. Phân hệ Student

Đăng ký tài khoản: Người dùng có thể tạo tài khoản bằng họ tên, email và mật khẩu. Backend phải kiểm tra email đã tồn tại hay chưa trước khi tạo tài khoản. Mật khẩu phải được mã hóa trước khi lưu vào Database.

Đăng nhập và đăng xuất: Người dùng đăng nhập bằng email và mật khẩu. Sau khi xác thực thành công, hệ thống cấp JWT có thời hạn sử dụng. Frontend phải duy trì trạng thái đăng nhập và cung cấp chức năng đăng xuất.

Trang danh sách khóa học: Trang chủ hiển thị danh sách khóa học dạng Card, gồm tên, danh mục, giảng viên, mô tả ngắn, học phí, số số tối đa, số lượng đã đăng ký và trạng thái Còn chỗ/Hết chỗ. Người dùng có thể tìm kiếm theo tên và lọc theo danh mục.



Trang chi tiết khóa học: Người dùng có thể xem đầy đủ thông tin khóa học và thực hiện ghi danh. Nếu chưa đăng nhập, hệ thống phải chuyển hướng đến trang Login trước khi tiếp tục thao tác.

Ghi danh khóa học: Backend phải kiểm tra học viên đã đăng ký khóa học hay chưa và kiểm tra capacity trước khi tạo Enrollment. Một học viên không được đăng ký cùng một khóa học nhiều lần. Nếu khóa học đã đạt capacity, hệ thống phải từ chối yêu cầu và trả về thông báo phù hợp.

Đối với trường hợp nhiều học viên cùng đăng ký suất cuối, hệ thống phải hạn chế Race Condition bằng Database Transaction, Row Lock, Atomic Update hoặc cơ chế phù hợp với Database/ORM được lựa chọn. Sau khi ghi danh thành công, dữ liệu Enrollment và số lượng học viên phải được đảm bảo nhất quán.

Khóa học của tôi: Student có thể xem danh sách các khóa học đã ghi danh, ngày ghi danh và trạng thái đăng ký. Người dùng chỉ được xem dữ liệu thuộc tài khoản của mình.

3.2. Phân hệ Admin / Staff

Đăng nhập Admin: Admin/Staff sử dụng tài khoản quản trị để truy cập hệ thống. Các route và API dành cho Admin phải được bảo vệ bằng JWT và kiểm tra role. Người dùng có role Student không được phép truy cập các chức năng quản trị.

Quản lý khóa học: Admin/Staff có quyền thêm mới, chỉnh sửa, xóa hoặc ẩn khóa học và cấu hình sĩ số tối đa. Backend phải kiểm tra tính hợp lệ của dữ liệu trước khi lưu. Capacity không được thiết lập thấp hơn số lượng học viên đã ghi danh.

Danh sách ghi danh: Admin/Staff có thể chọn từng khóa học để xem danh sách học viên đã đăng ký, gồm họ tên, email, tên khóa học, thời gian đăng ký và trạng thái.

Trạng thái giao diện: Frontend phải xử lý Loading, Empty State, Error State và Disabled State trong quá trình gọi API. Các thông báo thành công hoặc thất bại phải sử dụng Toast Notification hoặc UI tương đương, không sử dụng alert() mặc định.

4. YÊU CẦU KỸ THUẬT BẮT BUỘC

Frontend sử dụng React.js với Vite hoặc Next.js và React Router v6+ có Protected Routes. Giao diện phải Responsive trên Desktop và Mobile, đồng thời có đầy đủ UX Feedback như Loading Spinner, Empty State và Disabled Button khi submit.



Backend sử dụng Node.js với Express, Fastify hoặc NestJS để xây dựng RESTful API. Database có thể sử dụng MySQL, PostgreSQL hoặc MongoDB kết hợp với Prisma, Drizzle, TypeORM, Mongoose hoặc công nghệ tương đương. Hệ thống tối thiểu phải có ba thực thể Users, Courses và Enrollments.

Nghiệp vụ cốt lõi phải chặn đăng ký trùng lặp và chặn ghi danh khi enrolled_count >= capacity. Khuyến khích sử dụng Database Transaction để xử lý Race Condition khi nhiều người cùng ghi danh suất cuối. Ngày tháng sử dụng định dạng YYYY-MM-DD theo múi giờ Asia/Ho_Chi_Minh và API phải có cấu trúc JSON lỗi thống nhất.

Về bảo mật, mật khẩu phải được mã hóa bằng bcrypt với salt tối thiểu 10, JWT phải có thời hạn sử dụng và thông tin bí mật không được commit lên Repository. Project phải có file .env.example. Git history cần sử dụng Conventional Commits như feat, fix, refactor và không được bulk dump toàn bộ source code trong một commit.

5. THANG ĐIỂM ĐÁNH GIÁ 10 ĐIỂM

Hạng mục	Điểm	Tiêu chí mức Intern	Tiêu chí nâng cao
Backend & CSDL	3.0	ERD hợp lý (0.75đ); REST API CRUD & Auth (1.25đ); xử lý trùng lặp và sức chứa (1.0đ).	Transaction chống Race Condition; Input Validation; xử lý Date đúng yêu cầu.
Frontend & UI/UX	2.5	Responsive (1.0đ); gọi API danh sách & chi tiết (1.0đ); Login/Logout (0.5đ).	Đầy đủ Loading/Empty/Disabled; Toast; Code splitting.
Phân quyền & Bảo mật	1.5	Tách biệt Admin/Student (0.75đ); Protected Routes & JWT Middleware (0.75đ).	Bcrypt salt >= 10; RBAC; bảo mật .env.
Kiểm thử Postman	1.5	Postman Collection đầy đủ luồng (0.75đ); ảnh test CRUD và mã lỗi (0.75đ).	Unit/Integration Test bằng Jest, Vitest hoặc Supertest (+0.5đ bonus).
Git & Clean	1.5	Cấu trúc source rõ ràng (0.75đ);	Conventional Commits;



Code		README chuyên nghiệp (0.75đ).	.env.example; không bulk dump source.
------	--	-------------------------------	---------------------------------------

6. YÊU CẦU KIỂM THỬ API & POSTMAN: Ứng viên bắt buộc xây dựng và export Postman Collection chứa các API chính của hệ thống, bao gồm đăng ký, đăng nhập, quản lý khóa học và ghi danh. File Postman đặt trong thư mục /docs. Ứng viên cần kiểm thử các trường hợp thành công và các lỗi quan trọng, đồng thời cung cấp hình ảnh kết quả kiểm thử thể hiện rõ request và response. Khuyến khích triển khai thêm Unit Test hoặc Integration Test cho các nghiệp vụ quan trọng.

7. GIT & CLEAN CODE: Source code phải được tổ chức rõ ràng, dễ đọc và dễ bảo trì. Project cần có README hướng dẫn cài đặt, cấu hình Database, chạy Frontend, Backend và tài khoản Demo. Ứng viên phải sử dụng Git trong quá trình phát triển, không dồn toàn bộ source code vào một commit duy nhất và không commit secret lên Repository.

8. CẤU TRÚC BÀI NỘP: Project tối thiểu gồm Frontend, Backend và thư mục /docs chứa ERD, Postman Collection và hình ảnh kiểm thử. Ứng viên gửi GitHub Repository và link Demo hoặc Video thao tác nếu có, đồng thời cung cấp tài khoản Demo để người chấm kiểm tra các chức năng Student và Admin.

Lưu ý quan trọng về quy trình Nộp Bài Test:

Hình thức nộp bài quy định: Bạn **BẮT BUỘC** thực hiện tải đề bài, làm bài và gửi bài làm (upload file bài làm / đính kèm link GitHub / Demo) trực tiếp trên **Cổng nộp bài định danh hệ thống** thông qua liên kết ở trên.

Hệ thống tự động ghi nhận: Kết quả bài test chỉ được công nhận hợp lệ khi bạn hoàn thành thao tác bấm "**Nộp bài**" thành công trên Cổng nộp bài trực tuyến trong thời hạn **36 giờ** (tính từ thời điểm bạn bấm bắt đầu làm bài). **Hệ thống không tiếp nhận bài làm nộp qua Email cá nhân hoặc các kênh khác.**

Thời hạn nộp bài: Trường hợp không hoàn thành và nộp bài test trong thời hạn quy định, chúng tôi sẽ hiểu rằng bạn đã từ chối tiếp tục tham gia các vòng tuyển dụng tiếp theo.